

In addition to the following information, please see the supplementary video and project website (<https://mobilevista.github.io>) for a visual walkthrough of the framework and example rollout videos from each policy on each task.

A. Visual Observation Synthesis

1) *Robot Model and Segmentation*: We implement visual observation synthesis using MuJoCo via the robosuite framework. For each task, we render the robot using the same MuJoCo model used to instantiate the source demonstration, with non-robot geoms in the scene rendered invisible to produce a clean robot silhouette.

To extract a 2D mask of the robot, we use a different procedure across the simulated and real-world settings. In simulated experiments, we modify the scene’s skybox color to a distinctive background color (red for the GR-1 humanoid setting, yellow for the bimanual Franka setting, the latter chosen to avoid conflicts with red environment objects), render the robot with all other geoms hidden, and threshold pixels by background color to produce a binary mask. In the real-world setting, we instead render the R1 Pro robot model in MuJoCo from a URDF specification and extract the robot silhouette directly from the resulting depth buffer (thresholded at 0.995 to exclude background), then dilate the resulting mask by 5 pixels to capture edge artifacts.

We identify robot geoms in MuJoCo by body name prefixes: `robot0/gripper0` (and `robot1/gripper1` in the bimanual setting) for simulation, and the robot’s named body geoms (e.g., `left_arm_link*`, `torso_link*`) for the real-world R1 Pro setup. Rendering is performed at 512×512 resolution for simulated experiments and at the native dataset resolution for real-world experiments, upsampled by a factor of $6\times$ during point cloud reprojection.

2) *Depth Reprojection and Camera Extrinsic*s: We lift the masked RGB-D observation into a colored point cloud using a pinhole camera model with intrinsics derived from the camera’s vertical field of view (`cam_fovy` in MuJoCo). For real-world data, we use the ZED2 camera’s calibrated intrinsics ($f_x = 58.4$, $f_y = 58.5$, $c_x = 74.7$, $c_y = 42.9$ at the dataset resolution, scaled with image dimensions during upsampling). Point clouds are constructed using Open3D’s `RGBDImage.create_from_color_and_depth` with depth scale 1000mm and a depth truncation of 5m (simulation) or 10m (real).

To lift the point cloud into the world frame, we use the camera-to-world extrinsics obtained from MuJoCo at the canonical configuration (`cam_xpos`, `cam_xmat`). After applying the base perturbation Δp , we re-render the robot model at its retargeted joint configuration and obtain new camera extrinsics from the resulting kinematic chain. We reproject the world-frame point cloud onto the new camera using an Open3D `OffscreenRenderer`, producing a pose-shifted RGB image and a corresponding hole mask from pixels not covered by reprojected points.

3) *Inpainting*: To fill missing regions in the reprojected RGB image, including both regions originally occupied by the segmented-out robot and newly disoccluded areas introduced by the base shift, we apply video inpainting with E2FGVI [28]. We process each trajectory’s frames and per-frame hole masks as a temporal batch via E2FGVI’s official pipeline. Prior to inpainting, we apply a 3×3 morphological closing (3 iterations) to the hole mask to fill small gaps introduced by point cloud sparsity. We use the model checkpoint `E2FGVI-HQ-CVPR22.pth`.

4) *Robot Re-Compositing*: After inpainting, we re-render the robot model at its retargeted joint configuration using MuJoCo, with non-robot geoms hidden as before. We composite the re-rendered robot onto the inpainted background by masking robot pixels from the rendered image. In the simulation pipeline, we use a color-based mask (excluding pixels matching the masking skybox color) to handle background bleed-through. In the real-world pipeline, we use the depth buffer from the re-rendered robot to identify robot pixels, again dilated by 5 pixels to capture edges. After compositing, real-world outputs are downsampled back to the original dataset resolution.

B. Action Retargeting

1) *Forward and Inverse Kinematics*: In the simulated humanoid (GR-1) and bimanual Franka settings, actions are originally represented as per-arm end-effector poses (3D position + 3D rotation vector per arm) in the robot base frame. Retargeting is performed in closed form by applying the inverse base perturbation transform to each arm’s end-effector goal at each timestep (Section B.2). In the bimanual setting, the rig caches both robot base poses and the shared camera pose; the yaw rotation is applied about the midpoint of the two base positions to preserve their relative configuration.

In the real-world (R1 Pro) setting, we instead solve for new joint configurations using whole-body inverse kinematics. We use the `mink` [34] IK solver with the following setup:

- **FrameTasks** for the left and right end-effector sites (`left_ee_center`, `right_ee_center`), each with position cost and orientation cost set to 1.0.
- **PostureTask** regularizing toward the previous configuration, with cost 10^{-5} .
- **ConfigurationLimit** on joint positions.
- **VelocityLimit** with a per-step bound of 1.0 on arm joints. The torso joint limit is set to 0.02 for Book Stacking and 0.04 for Laundry Sort, preferring arm motion over torso motion to maintain a stable view-point while allowing slightly more torso motion for the longer-horizon Laundry Sort task.
- **CollisionAvoidanceLimit** between end-effector and torso geom groups with a minimum collision distance of 0.04m and a detection distance of 0.18m.

We iterate the IK solver for up to 20 steps per frame with $\Delta t = 0.01$ using `quadprog` as the underlying solver, with damping 10^{-3} and a position tolerance of 0.01m for convergence.

2) *End-Effector Goal Perturbation*: Given a base perturbation $\Delta p = (\Delta b, \Delta \theta)$ with translation $\Delta b \in \mathbb{R}^2$ and yaw $\Delta \theta$, we construct the inverse transform $T_{\Delta p}^{-1}$ as a 4×4 matrix combining the inverse yaw rotation and a translation. For each arm’s per-timestep end-effector position $\mathbf{p}_t$ (in the robot’s base frame), the compensated position is $T_{\Delta p}^{-1} \cdot [\mathbf{p}_t; 1]$. Orientations, represented as rotation vectors, are updated by composing with the inverse yaw rotation: $R'_t = R_{\text{inv yaw}} \cdot R(\mathbf{r}_t)$, then converted back to a rotation vector. Gripper and finger commands are passed through unchanged.

For the bimanual Franka setting, the retargeting accounts for each arm’s new base position after yaw is applied about the rig midpoint, so the per-arm compensation includes both a rotation and a translation derived from the change in that arm’s base pose.

For the real-world R1 Pro setting, the end-effector position and orientation targets passed to IK at $t = 0$ are obtained by transforming the original end-effector poses around the rig pivot under Δp . For subsequent timesteps, IK is solved to track the original end-effector trajectory directly, allowing the arm to bend differently to reach the same world-frame target from the new base pose.

C. Augmentation Sampling

For each augmented trajectory, we sample a base translation offset Δb uniformly within a disk of radius $r_{\text{train}} = 15\text{cm}$ centered at the canonical pose using the standard formulation $(r, \theta) \sim (\sqrt{u} \cdot r_{\text{train}}, 2\pi v)$ for $u, v \sim \mathcal{U}(0, 1)$, with $\Delta b = (r \cos \theta, r \sin \theta, 0)$. No vertical offset is applied. When yaw jitter is enabled, we additionally sample $\Delta \theta \sim \mathcal{U}(-5^\circ, 5^\circ)$. We use the same training and evaluation radius $r_{\text{train}} = r_{\text{eval}} = 15\text{cm}$ across all simulated and real-world tasks.

For simulated tasks, we generate one augmented trajectory per source demonstration, yielding 200 augmented trajectories from 200 source demonstrations. For real-world tasks, we generate ten augmented trajectories per source demonstration, yielding 500 augmented trajectories from 50 source demonstrations. We do not filter augmented trajectories based on IK convergence; in practice, the IK solver converges within tolerance for the vast majority of frames.

D. Task Definitions

We evaluate on four tasks from DexMimicGen [32]. For the Can Sort task, we use an even split of expert demonstrations across can color (blue, red) for training. For the remainder of the tasks, we train using the first 200 demonstrations of the datasets and evaluate using the environment definitions provided by DexMimicGen, detailed below. Can Sort and Pouring are performed on a Fourier GR-1 humanoid, and Box and Drawer are performed on a bimanual Franka setup.

- **Can Sort**: The robot must pick up a can with one arm, perform a handover to the other arm, and deposit it into the correct bin based on color. The handover phase is a coordination subtask requiring both arms to cooperate with synchronized trajectories. Can color and placement

are randomized according to DexMimicGen defaults; the target bins are fixed.

- **Pouring**: A sequential task in which the robot must first pour a ball from a cup into a bowl using one arm, then pick up the bowl with the other arm and place it onto a green pad. The cup and bowl positions are randomized; the pad location is fixed.
- **Box**: The robot must pick up a box lid and place it onto the box using both arms simultaneously, requiring precise relative alignment between the two end-effectors. The positions of the box and lid are randomized.
- **Drawer**: A sequential task in which the robot must open a drawer with one arm before placing a mug inside with the other. The mug location is randomized according to DexMimicGen defaults.

Camera setup. For the bimanual Franka tasks (Box, Drawer), we use the `agentview` camera, a fixed rigid camera positioned between the two arms to approximate an egocentric viewpoint. For the GR-1 humanoid tasks (Can Sort, Pouring), we use the `robot0_robotview` head-mounted camera. We widen its field of view from the DexMimicGen default of 50° to 90° to ensure the full manipulation workspace is visible.

E. Tabled Per-Task Results

Randomized Pose Perturbations. Table III reports full per-task success rates for all methods under the randomized base pose perturbation regime from Section IV-A (Figure 3) of the main paper. Each cell reports mean success rate and standard deviation over three random seeds.

Ablations. Table IV reports full per-task ablation results corresponding to Figure 4 of the main paper. Each cell reports mean $\pm$ std over three random seeds. MobileVISTA wins across nearly all tasks and perturbation magnitudes. The exception is the Drawer task, where Fixed Robot Geom. is competitive or superior at intermediate perturbation radii, suggesting that this task may be less sensitive to inaccurate robot re-rendering.

F. Comparison of MobileVISTA with Learned Novel View Synthesis

We provide additional details supporting the comparison between MobileVISTA and the learned novel view synthesis baseline GEN3C [11] introduced in Section IV-C. We report quantitative reconstruction quality, qualitative side-by-side comparisons, generation throughput, and downstream policy learning performance when training on views synthesized by each method.

Data Generation. We integrate GEN3C [11] into our pipeline as a drop-in replacement for the geometry-based reprojection step in MobileVISTA, while keeping the action retargeting and robot re-compositing components unchanged. To ensure a fair comparison, we make several modifications to the default GEN3C setup.

First, rather than estimating depth and camera parameters from RGB using VIPE as in the official GEN3C implementation, we seed GEN3C with ground-truth depth and ground-

Task	Method	$r=0$	$r=1$	$r=3$	$r=5$	$r=7$	$r=10$	$r=15$
Can Sort	Canonical-2D	0.480 $\pm$ 0.02	0.500 $\pm$ 0.09	0.393 $\pm$ 0.04	0.440 $\pm$ 0.05	0.327 $\pm$ 0.03	0.240 $\pm$ 0.00	0.120 $\pm$ 0.06
	DP3	0.733 $\pm$ 0.08	0.713 $\pm$ 0.06	0.600 $\pm$ 0.13	0.407 $\pm$ 0.05	0.327 $\pm$ 0.06	0.153 $\pm$ 0.04	0.060 $\pm$ 0.02
	Fine-tuned $\pi_{0.5}$	0.926 $\pm$ 0.05	0.906 $\pm$ 0.01	0.833 $\pm$ 0.06	0.827 $\pm$ 0.10	0.780 $\pm$ 0.09	0.607 $\pm$ 0.09	0.320 $\pm$ 0.03
	MobileVISTA (Ours)	0.953$\pm$0.03	0.967$\pm$0.02	0.933$\pm$0.02	0.887$\pm$0.05	0.873$\pm$0.05	0.793$\pm$0.06	0.527$\pm$0.04
	<i>Simulator (Oracle)</i>	0.913 $\pm$ 0.02	0.927 $\pm$ 0.05	0.893 $\pm$ 0.01	0.927 $\pm$ 0.03	0.873 $\pm$ 0.05	0.727 $\pm$ 0.01	0.527 $\pm$ 0.08
Pouring	Canonical-2D	0.547 $\pm$ 0.03	0.413 $\pm$ 0.01	0.460 $\pm$ 0.08	0.353 $\pm$ 0.03	0.200 $\pm$ 0.09	0.080 $\pm$ 0.02	0.073 $\pm$ 0.06
	DP3	0.287 $\pm$ 0.08	0.240 $\pm$ 0.09	0.247 $\pm$ 0.06	0.173 $\pm$ 0.06	0.113 $\pm$ 0.03	0.080 $\pm$ 0.06	0.013 $\pm$ 0.01
	Fine-tuned $\pi_{0.5}$	0.467 $\pm$ 0.03	0.447 $\pm$ 0.02	0.433 $\pm$ 0.08	0.333 $\pm$ 0.05	0.207 $\pm$ 0.06	0.093 $\pm$ 0.03	0.013 $\pm$ 0.01
	MobileVISTA (Ours)	0.673$\pm$0.02	0.640$\pm$0.03	0.640$\pm$0.06	0.613$\pm$0.04	0.607$\pm$0.10	0.547$\pm$0.12	0.460$\pm$0.02
	<i>Simulator (Oracle)</i>	0.773 $\pm$ 0.03	0.787 $\pm$ 0.06	0.713 $\pm$ 0.13	0.727 $\pm$ 0.03	0.673 $\pm$ 0.03	0.700 $\pm$ 0.07	0.560 $\pm$ 0.11
Box	Canonical-2D	0.307 $\pm$ 0.11	0.273 $\pm$ 0.02	0.227 $\pm$ 0.09	0.147 $\pm$ 0.10	0.100 $\pm$ 0.03	0.047 $\pm$ 0.02	0.000 $\pm$ 0.00
	DP3	0.407 $\pm$ 0.01	0.473 $\pm$ 0.02	0.373 $\pm$ 0.05	0.333 $\pm$ 0.02	0.293 $\pm$ 0.05	0.213 $\pm$ 0.03	0.087 $\pm$ 0.05
	Fine-tuned $\pi_{0.5}$	0.187 $\pm$ 0.10	0.187 $\pm$ 0.06	0.187 $\pm$ 0.08	0.147 $\pm$ 0.01	0.107 $\pm$ 0.01	0.060 $\pm$ 0.03	0.020 $\pm$ 0.00
	MobileVISTA (Ours)	0.520$\pm$0.14	0.600$\pm$0.07	0.427$\pm$0.08	0.440$\pm$0.07	0.413$\pm$0.03	0.393$\pm$0.07	0.247$\pm$0.05
	<i>Simulator (Oracle)</i>	0.313 $\pm$ 0.09	0.373 $\pm$ 0.02	0.433 $\pm$ 0.10	0.407 $\pm$ 0.06	0.473 $\pm$ 0.05	0.373 $\pm$ 0.06	0.333 $\pm$ 0.01
Drawer	Canonical-2D	0.287 $\pm$ 0.10	0.260 $\pm$ 0.05	0.280 $\pm$ 0.00	0.100 $\pm$ 0.04	0.120 $\pm$ 0.03	0.040 $\pm$ 0.02	0.013 $\pm$ 0.01
	DP3	0.260 $\pm$ 0.00	0.273 $\pm$ 0.02	0.220 $\pm$ 0.05	0.180 $\pm$ 0.03	0.093 $\pm$ 0.05	0.067 $\pm$ 0.06	0.040 $\pm$ 0.02
	Fine-tuned $\pi_{0.5}$	0.580 $\pm$ 0.09	0.513 $\pm$ 0.05	0.400 $\pm$ 0.02	0.373 $\pm$ 0.11	0.227 $\pm$ 0.06	0.167 $\pm$ 0.08	0.060 $\pm$ 0.02
	MobileVISTA (Ours)	0.653$\pm$0.04	0.620$\pm$0.05	0.580$\pm$0.07	0.593$\pm$0.06	0.587$\pm$0.02	0.487$\pm$0.09	0.413$\pm$0.01
	<i>Simulator (Oracle)</i>	0.720 $\pm$ 0.03	0.780 $\pm$ 0.07	0.627 $\pm$ 0.03	0.713 $\pm$ 0.07	0.567 $\pm$ 0.05	0.640 $\pm$ 0.09	0.393 $\pm$ 0.03

TABLE III: Detailed per-task success rates under randomized pose perturbation across evaluation radii $r_{eval} \in \{0, 1, 3, 5, 7, 10, 15\}$ cm with yaw $\Delta\theta \sim \mathcal{U}(-5^\circ, 5^\circ)$. Each cell reports the mean $\pm$ std over three random seeds (50 evaluation rollouts per seed). **Bold** indicates the best non-oracle method per row. Simulator (Oracle) is reported as a reference upper bound.

Task	Method	$r=0$	$r=1$	$r=3$	$r=5$	$r=7$	$r=10$	$r=15$
Can Sort	Canonical-2D	0.48 $\pm$ 0.02	0.50 $\pm$ 0.09	0.39 $\pm$ 0.04	0.44 $\pm$ 0.05	0.33 $\pm$ 0.03	0.24 $\pm$ 0.00	0.12 $\pm$ 0.06
	Aug. Images Only	0.54 $\pm$ 0.05	0.45 $\pm$ 0.09	0.36 $\pm$ 0.00	0.17 $\pm$ 0.10	0.11 $\pm$ 0.04	0.03 $\pm$ 0.02	0.03 $\pm$ 0.02
	Aug. Actions Only	0.08 $\pm$ 0.07	0.11 $\pm$ 0.08	0.09 $\pm$ 0.01	0.07 $\pm$ 0.06	0.07 $\pm$ 0.05	0.07 $\pm$ 0.04	0.07 $\pm$ 0.05
	Fixed Robot Geom.	0.80 $\pm$ 0.05	0.80 $\pm$ 0.07	0.81 $\pm$ 0.05	0.64 $\pm$ 0.05	0.46 $\pm$ 0.09	0.31 $\pm$ 0.10	0.17 $\pm$ 0.06
	MobileVISTA (Ours)	0.95 $\pm$ 0.03	0.97 $\pm$ 0.02	0.93 $\pm$ 0.02	0.89 $\pm$ 0.05	0.87 $\pm$ 0.05	0.79 $\pm$ 0.06	0.53 $\pm$ 0.04
Pouring	Canonical-2D	0.55 $\pm$ 0.03	0.41 $\pm$ 0.01	0.46 $\pm$ 0.08	0.35 $\pm$ 0.03	0.20 $\pm$ 0.09	0.08 $\pm$ 0.02	0.07 $\pm$ 0.06
	Aug. Images Only	0.40 $\pm$ 0.05	0.41 $\pm$ 0.04	0.33 $\pm$ 0.12	0.31 $\pm$ 0.08	0.08 $\pm$ 0.02	0.04 $\pm$ 0.04	0.05 $\pm$ 0.02
	Aug. Actions Only	0.00 $\pm$ 0.00	0.01 $\pm$ 0.01	0.00 $\pm$ 0.00	0.01 $\pm$ 0.01	0.01 $\pm$ 0.01	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00
	Fixed Robot Geom.	0.31 $\pm$ 0.08	0.25 $\pm$ 0.01	0.16 $\pm$ 0.04	0.10 $\pm$ 0.07	0.04 $\pm$ 0.02	0.03 $\pm$ 0.01	0.01 $\pm$ 0.02
	MobileVISTA (Ours)	0.67 $\pm$ 0.02	0.64 $\pm$ 0.03	0.64 $\pm$ 0.06	0.61 $\pm$ 0.04	0.61 $\pm$ 0.10	0.55 $\pm$ 0.12	0.46 $\pm$ 0.02
Box	Canonical-2D	0.31 $\pm$ 0.11	0.27 $\pm$ 0.02	0.23 $\pm$ 0.09	0.15 $\pm$ 0.10	0.10 $\pm$ 0.03	0.05 $\pm$ 0.02	0.00 $\pm$ 0.00
	Aug. Images Only	0.15 $\pm$ 0.07	0.13 $\pm$ 0.08	0.05 $\pm$ 0.01	0.02 $\pm$ 0.00	0.01 $\pm$ 0.01	0.01 $\pm$ 0.01	0.00 $\pm$ 0.00
	Aug. Actions Only	0.01 $\pm$ 0.01	0.01 $\pm$ 0.01	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.01 $\pm$ 0.01	0.00 $\pm$ 0.00
	Fixed Robot Geom.	0.22 $\pm$ 0.07	0.25 $\pm$ 0.09	0.23 $\pm$ 0.02	0.22 $\pm$ 0.11	0.22 $\pm$ 0.05	0.11 $\pm$ 0.03	0.11 $\pm$ 0.03
	MobileVISTA (Ours)	0.52 $\pm$ 0.14	0.60 $\pm$ 0.07	0.43 $\pm$ 0.08	0.44 $\pm$ 0.07	0.41 $\pm$ 0.03	0.39 $\pm$ 0.07	0.25 $\pm$ 0.05
Drawer	Canonical-2D	0.29 $\pm$ 0.10	0.26 $\pm$ 0.05	0.28 $\pm$ 0.00	0.10 $\pm$ 0.04	0.12 $\pm$ 0.03	0.04 $\pm$ 0.02	0.01 $\pm$ 0.01
	Aug. Images Only	0.12 $\pm$ 0.03	0.09 $\pm$ 0.01	0.07 $\pm$ 0.05	0.01 $\pm$ 0.01	0.01 $\pm$ 0.01	0.01 $\pm$ 0.01	0.00 $\pm$ 0.00
	Aug. Actions Only	0.01 $\pm$ 0.01	0.01 $\pm$ 0.02	0.01 $\pm$ 0.01	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.01 $\pm$ 0.01	0.00 $\pm$ 0.00
	Fixed Robot Geom.	0.55 $\pm$ 0.12	0.62 $\pm$ 0.09	0.64 $\pm$ 0.07	0.59 $\pm$ 0.13	0.66 $\pm$ 0.05	0.51 $\pm$ 0.08	0.36 $\pm$ 0.05
	MobileVISTA (Ours)	0.65 $\pm$ 0.04	0.62 $\pm$ 0.05	0.58 $\pm$ 0.07	0.59 $\pm$ 0.06	0.59 $\pm$ 0.02	0.49 $\pm$ 0.09	0.41 $\pm$ 0.01

TABLE IV: Detailed ablation results across tasks. Success rate (mean $\pm$ std over three seeds) vs. evaluation radius $r_{eval} \in \{0, 1, 3, 5, 7, 10, 15\}$ cm with yaw $\Delta\theta \sim \mathcal{U}(-5^\circ, 5^\circ)$. **Bold** indicates the best method per column within each task.

truth camera intrinsics and extrinsics from the simulator. This removes a source of error that is orthogonal to the central question of how well each method synthesizes pose-shifted observations.

Second, we assume access to the same robot model used by MobileVISTA when rendering the perturbed scene from the new camera trajectory. This is necessary because, as discussed in Section III-B, a base perturbation induces a

corresponding change in the egocentric camera trajectory through the base-to-camera kinematic chain after action retargeting, and the new camera trajectory cannot be computed without the robot model. Sharing this information with GEN3C ensures both methods use the same target camera trajectory for synthesis.

Finally, GEN3C generates video in fixed 121-frame chunks. We therefore generate each augmented trajectory in

chunks of $121 \times 3 = 363$ frames; trajectories longer than this are truncated at the end. In practice, this truncation affected at most 12 frames per trajectory across the four tasks evaluated, and truncation always occurred at the trailing end of the trajectory.

PSNR Comparison. To quantify reconstruction quality, we render oracle egocentric views from the simulator at perturbed base poses and treat these as ground truth. We then synthesize views at the same perturbed poses using MobileVISTA and GEN3C and compute PSNR against the oracle. For each task, we sample 200 perturbed configurations within a disk of radius $r = 15\text{cm}$ around the canonical pose. Table V reports the mean PSNR across these configurations for each task.

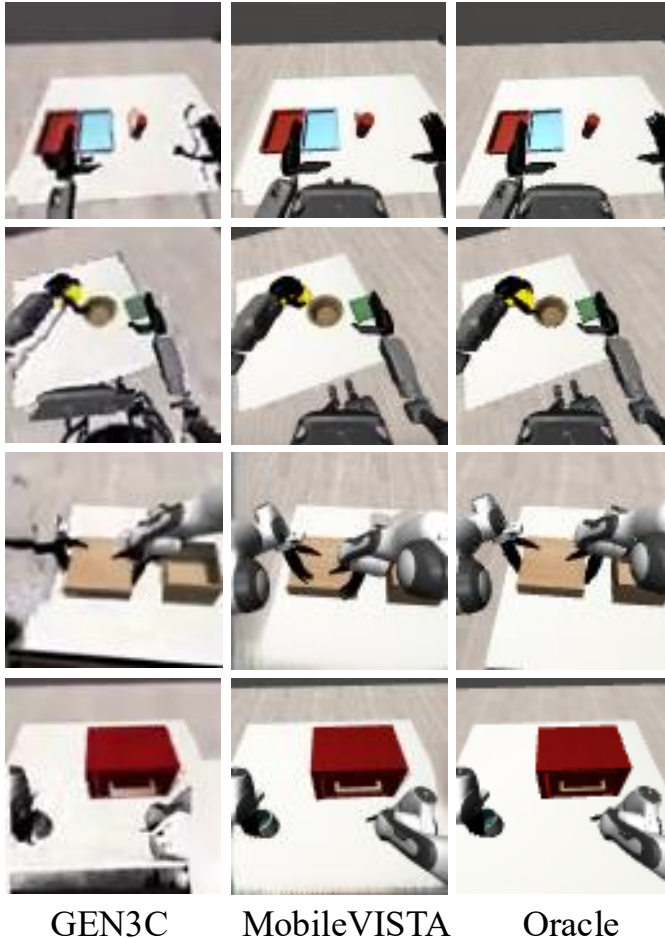


Fig. 8: Qualitative comparison of synthesized egocentric observations. For each task, we show the simulator oracle view, the MobileVISTA-synthesized view, and the GEN3C-synthesized view at the same perturbed base pose. MobileVISTA more faithfully preserves scene geometry and object structure.

MobileVISTA produces higher PSNR than GEN3C across all four tasks, with the largest gaps appearing on the humanoid tasks (Can Sort and Pouring). The trends in Table V are illustrated in the per-trajectory scatter

	GEN3C	M.Vista
Can Sort	14.93	3.81
Pouring	13.65	4.31
Box	14.93	2.39
Drawer	16.85	2.99

TABLE VI: $\downarrow$ Avg. synthesis time per trajectory (minutes). MobileVISTA is 4-7 $\times$ faster than GEN3C across tasks.

visualization in Figure 6 of the main paper.

Qualitative Comparisons. Figure 8 shows representative synthesized observations from MobileVISTA and GEN3C alongside the simulator oracle at matched perturbed base poses. MobileVISTA preserves sharp object boundaries and consistent scene geometry across perturbations, while GEN3C tends to blur object edges and distort surfaces under larger viewpoint shifts. These visual differences are consistent with the PSNR gap reported in Table V.

Data Generation Speed. We measure the wall-clock time required to synthesize one augmented trajectory using each method, averaged across 200 trajectories per task. MobileVISTA timings are measured on a single NVIDIA L40S GPU, and GEN3C timings are measured on a single NVIDIA H200 GPU. Table VI reports per-task averages. MobileVISTA generates augmented trajectories 4-7 $\times$ faster than GEN3C across tasks. Note that as the H200 is a higher-end GPU than the L40S, so the reported speed advantage of MobileVISTA is a conservative estimate of the true gap on matched hardware. We postulate this gap arises as GEN3C performs iterative diffusion sampling for each frame, while MobileVISTA relies on a single forward pass through the geometry-based reprojection pipeline followed by inpainting.

G. Policy Learning from GEN3C-Synthesized Views

We additionally evaluate downstream policy performance when training on GEN3C-synthesized views rather than MobileVISTA-synthesized views. To isolate the effect of the visual synthesis method, we pair GEN3C-synthesized observations with the same action retargeting procedure described in Section III-B, and train diffusion policies using the same configuration as MobileVISTA. We evaluate under the same randomized perturbation regime as Section IV-A.

Results are shown in Figure 5. The Canonical-2D, Simulator (Oracle), and MobileVISTA results are reproduced from Section IV-A for reference. Policies trained on MobileVISTA-synthesized data outperform those trained on GEN3C-synthesized data across all four tasks and across the full range of evaluation perturbation magnitudes. We find the gap in downstream performance is consistent with the reconstruction quality gap reported in Table V: tasks where the two methods produce more similar PSNR also generally show smaller gaps in policy success rate.

H. Hardware Setup

We deploy MobileVISTA and baselines on a Galaxea R1 Pro humanoid robot. The robot is equipped with a head-mounted ZED2 RGB-D camera providing the egocentric view used by all policies, and two wrist-mounted Intel RealSense RGB cameras.

To enable repeatable evaluation across base pose perturbations, we manually mark a grid on the floor extending 15cm in each direction from the canonical robot base pose. Markings are placed by hand and verified against ground truth measurements before evaluation begins. During evaluation, we manually drive the robot to the sampled offset

Method	Can Sort			Pouring			Box			Drawer		
	0-5cm	5-10cm	10-15cm	0-5cm	5-10cm	10-15cm	0-5cm	5-10cm	10-15cm	0-5cm	5-10cm	10-15cm
GEN3C	16.29	14.42	13.08	17.74	15.09	13.28	16.37	13.75	11.78	16.32	15.76	14.35
M.Vista	25.02	24.05	23.28	23.71	23.25	22.18	16.47	16.01	14.72	18.67	18.93	18.29

TABLE V: $\uparrow$ PSNR (dB) between synthesized egocentric views and simulator oracle images across base perturbation magnitudes r .

using these markings as reference, ensuring consistent and repeatable starting configurations across rollouts.

All policy inference is performed on a single NVIDIA RTX 4090 GPU.

I. Data Collection & Real World Task Definitions

Teleoperation. We collect expert demonstrations using the JoyLo teleoperation framework introduced in BEHAVIOR Robot Suite [33]. Demonstrations are recorded in ROS 2 at 10Hz with synchronized RGB-D images, proprioceptive states, and joint-space actions including arm, torso, and gripper commands.

Book Stacking Task Definition. The robot must grasp a vertically placed book from a designated location and reorient it horizontally onto an existing stack inside a bookshelf. We collect 50 expert demonstrations from the canonical (non-perturbed) robot pose, each approximately 150 timesteps in length.

Laundry Sort Task Definition. The robot must sort four laundry items into two bins based on category: two black socks must be placed into a grey bin on the left, and a yellow towel and a beige pair of shorts must be placed into a larger green bin on the right. We do not apply domain randomization to the item placement or appearance. This task is substantially more challenging than Book Stacking due to its longer horizon, the deformability of the items, and the bimanual coordination required to sort multiple objects in sequence.

J. Evaluation Protocol

For each trial, we sample a base translation offset uniformly within a disk of radius $r_{eval} = 15\text{cm}$ centered at the canonical pose. We manually drive the robot to that offset using the floor markings described in Section H. Yaw is not explicitly controlled, since policies are trained over the full $\pm 5^\circ$ yaw range used during augmentation.

From each sampled offset, we roll out all four policies (MobileVISTA, Canonical-2D, DP3, Fine-Tuned $\pi_{0.5}$) without repositioning the robot, enabling direct comparison from identical starting configurations. The order in which policies are evaluated at each offset is randomized and unknown to the experimenter to eliminate bias.

For Book Stacking, we score each rollout in two stages: Grasp, where the robot successfully grasps the book, and Stacked, where the book is successfully placed onto the existing stack. We perform 25 trials at the canonical pose and 25 trials at randomized poses, for a total of 50 trials per policy.

For Laundry Sort, we score each rollout by the number of items correctly sorted into the appropriate bin, ranging from

0 to 4. We perform 25 trials at the canonical pose and 25 trials at randomized poses, for a total of 50 trials per policy.

K. MobileVISTA and Canonical-2D Diffusion Policy

We train MobileVISTA and Canonical-2D policies using Diffusion Policy [29] within the Robomimic framework [35], using the official implementation¹. All hyperparameters are identical across all tasks and all policy variants trained via this pipeline (MobileVISTA, Canonical-2D, Aug. Images Only, Aug. Actions Only, Fixed Robot Geom.). Full hyperparameters are listed in Table VII. For simulated tasks we train for 1400 epochs. For the real-world Book Stacking task we train for 3500 epochs, and for the real-world Laundry task we train for 6500 epochs, where each epoch consists of 500 gradient steps.

The proprioceptive observations differ between simulation and real. In simulation, we use end-effector positions and quaternions for each arm, along with gripper joint positions. In the real-world setting, we instead use joint positions for the left and right arms, torso joint positions, and binary gripper open signals. All three RGB cameras (egocentric and two wrist cameras) are used in both settings.

L. DP3

We train DP3 [30] using the official implementation² with default hyperparameters. Key settings include: observation horizon 2, action horizon 8, prediction horizon 16, DDIM noise scheduler with 100 training steps and 10 inference steps, AdamW optimizer (lr $1e-4$, weight decay $1e-6$) with cosine scheduling and 500 warmup steps, batch size 128, and EMA (power 0.75). Point clouds are encoded with a PointNet encoder (output dim 64) using color as an additional input channel regardless of task. For simulated tasks we train for 1400 epochs. Due to compute constraints, real-world policies are trained by global step count rather than epochs: 634,845 steps for Book Stacking and 1.3M steps for Laundry. We train three random seeds per task for all simulated experiments.

M. Fine-Tuned $\pi_{0.5}$

We fine-tune $\pi_{0.5}$ [1] using the official openpi repository³, initializing from the released pi05_base checkpoint. We use LoRA fine-tuning (gemma.2b.lora for the vision-language backbone and gemma.300m.lora for the action expert), with EMA disabled. All other training settings follow

¹https://github.com/real-stanford/diffusion_policy

²<https://github.com/YanjieZe/3D-Diffusion-Policy>

³<https://github.com/Physical-Intelligence/openpi>

Hyperparameter	Value
<i>Optimization</i>	
Learning rate	1e-4
L2 regularization	0.0
Batch size	16
Steps per epoch	500
Action normalization	min-max
<i>Policy horizons</i>	
Observation horizon	2
Action horizon	8
Prediction horizon	16
<i>Diffusion</i>	
Noise scheduler	DDIM
Training diffusion steps	100
Inference diffusion steps	10
Beta schedule	squaredcos_cap_v2
Prediction type	ϵ -prediction
EMA power	0.75
<i>UNet</i>	
Channel dims	[256, 512, 1024]
Kernel size	5
Groups	8
Diffusion step embed dim	256
<i>Image encoder</i>	
Backbone	ResNet-18 (no pretraining)
Pooling	SpatialSoftmax (32 keypoints)
Feature dim per camera	64
Input resolution	84×84
Random crop	76×76

TABLE VII: Diffusion Policy hyperparameters used for all MobileVISTA and Canonical-2D policy variants across simulated and real-world tasks.

the `pi05_libero` reference config in the `openpi` repository, with `action_dim=32` and `action_horizon=16`.

Simulated experiments. For each task, we train three random seeds for up to 30,000 steps, saving a checkpoint every 1,000 steps. For each seed, we sweep over all saved checkpoints by evaluating 15 rollouts at each of three perturbation radii ($r \in \{0, 5, 10\}$ cm) and selecting the checkpoint with the highest mean success rate across all three radii. We then run full evaluation (50 rollouts per radius, all perturbation magnitudes) on the selected checkpoint for each seed, and report results averaged over the three seeds.

Real-world experiments. For the real-world Book Stacking task, we average the selected checkpoint steps across the three best-performing simulated seeds to obtain a single fine-tuning step count of 22,000 steps. For the real-world Laundry task, we scale this by the ratio of Robomimic diffusion policy training durations between the two real-world tasks ($6500/3500 \approx 1.86\times$), yielding 41,000 steps. This scaling aligns the relative fine-tuning budget of $\pi_{0.5}$ with that of the Robomimic-based diffusion policies across the two tasks.